Réf : …../…… AU : 2026-2027

Université de Sousse

Ecole Nationale d'Ingénieurs de Sousse

Rapport de Stage d'été

[ ] Initiation [X]Ingénieur

Réalisé par :

GHAZEL Sahar

Filière :

Génie électronique industrielle

Stage effectué au sein de Novation City

Novation City, Sousse

Période du stage : 29/06/2026 - 07/08/2026

Remerciements

Avant d'entamer ce rapport, je tiens à exprimer ma sincère gratitude envers toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de mon stage d'été.

Je remercie tout d'abord M. Mohamed Chrifa, mon encadrant industriel au sein de Novation City, pour son accueil chaleureux, sa disponibilité et ses précieux conseils tout au long de ce stage. Son accompagnement m'a permis de progresser efficacement dans la réalisation de mon projet et de mieux appréhender les exigences du milieu professionnel dans le domaine de l'Industrie 4.0.

Mes remerciements s'adressent également à l'ensemble de l'équipe de Novation City, Centre d'Excellence en Industrie 4.0 de Sousse, pour leur accueil chaleureux, leur esprit d'équipe et le cadre de travail stimulant qu'ils m'ont offert durant toute la période de mon stage.

Je souhaite également remercier l'ensemble du corps professoral et administratif de l'École Nationale d'Ingénieurs de Sousse pour la qualité de la formation dispensée tout au long de mon cursus, ainsi que pour le suivi assuré durant cette période de stage.

Sommaire

Fiche-synthèse du stage 4
Introduction générale 5
Chapitre 1 : Organisme d'accueil et problématique traitée 6
1.1. Introduction 6
1.2. Présentation de l'entreprise 6
1.2.1. Produits et services 7
1.2.2. Organisation générale de l'entreprise 7
1.3. Problématique traitée et travail demandé 8
1.4. Démarche-Méthodologie utilisée 8
1.5. Chronogramme des tâches menées durant le stage 9
1.6. Conclusion : Principales observations et constations 10
Chapitre 2 : Analyse des besoins et conception 11
2.1. Introduction 11
2.2. Etude de l'existant 12
2.3. Analyse (ou spécification) des besoins 13
2.4. Conception 15
2.5. Conclusion 17
Chapitre 3 : Réalisation 17
3.1. Introduction 17
3.2. Choix technologiques et environnements de développement 20
3.3. Les fonctionnalités réalisées 22
3.4. Les interfaces ou le prototype (éventuellement) 22
3.5. Tests et simulations (éventuellement) 23
3.6. Conclusion 25
Conclusion générale 26
Mon bilan de compétences 27
Références 28

Fiche-synthèse du stage

Nom d l'entreprise	Novation City
Ville	Sousse
Nom de l'encadrant industriel	Mohamed Chrifa
Date début et date fin du stage	29/06/2026 – 7/08/2026
Nombre de jours effectifs passés dans les locaux de l'entreprise	30 jours
Les principales tâches réalisées durant le stage (maximum 3)	- Conception et développement d'un pipeline de vision par ordinateur (MediaPipe, MobileNetV2) pour la détection de gestes de la main et le suivi de cycles de travail (prise → travail→ dépose) - Déploiement et portage du pipeline sur systèmes embarqués (Raspberry Pi 4 et Jetson Nano) - Développement d'un tableau de bord web (Flask) pour la supervision temps réel et l'historisation des cycles
Le stage contient-il une période d'observation ? si oui combien de jours ça a durée.	Oui, 5 jours consacrés à la découverte du centre Novation City, de ses équipements Industrie 4.0 et de la problématique et recherche du projet
Le stage contient-il une partie conception ? si oui combien de jours ça a pris	Oui, 15 jours (analyse du besoin, choix technologiques, conception de l'architecture du pipeline et de la machine à états IDLE → PIECE_DETECTEE → EN_COURS_TRAVAIL → TERMINE)
Le stage contient-il une partie développement ? si oui combien de jours ça a pris	Oui, 10 jours (développement du script principal, intégration d'EfficientDet-Lite0, déploiement embarqué et développement du tableau de bord Flask)
Quels sont les prérequis pour ce stage en termes de compétences techniques?	Programmation Python, notions de traitement d'image et de vision par ordinateur, bases en systèmes embarqués / Linux, notions de Edge IA
Les outils et logiciels utilisés durant le stage	Python, OpenCV, MediaPipe, TensorFlow Lite, ,Flask, Cuda,Miniforge/conda, FileZilla, PuTTY
Le matériel et la machinerie (autre que les PCs) utilisés durant le stage	Raspberry Pi 4, Jetson Nano (carte porteuse Auvidea JN30D),Jetson TX2,caméra webcam USB,
Méthodologies utilisés durant le stage	Développement incrémental avec validation étape par étape, modélisation par machine à états (IDLE → PIECE_DETECTEE → EN_COURS_TRAVAIL → TERMINE)
Les deux principales qualifications-compétences comportementales acquises à travers ce stage	Autonomie et rigueur dans la conduite d'un projet technique, organisation et gestion du temps sur un développement multiétape
Les deux principales qualifications-compétences techniques acquises à travers ce stage	Vision par ordinateur et déploiement de systèmes d'IA embarqués sur plateformes hétérogènes (Raspberry Pi, Jetson Nano)

Introduction générale

L'industrie 4.0 repose sur un principe simple mais exigeant : rendre chaque poste de travail visible, mesurable et pilotable en temps réel. Or, sur de nombreuses lignes de production, le suivi des opérateurs et de leurs cycles de travail reste encore réalisé manuellement, ce qui limite la précision des données de performance et la réactivité face aux anomalies. C'est dans ce contexte que s'inscrit mon stage d'été, réalisé au sein de Novation City, Centre d'Excellence en Industrie 4.0 à Sousse.

Étudiante en cycle ingénieur, filière Génie électronique industrielle à l'École Nationale d'Ingénieurs de Sousse, je souhaite orienter mon parcours vers les systèmes embarqués intelligents et la vision par ordinateur appliquée à l'industrie. Novation City, en tant que centre de référence régional pour la transformation digitale des entreprises, représentait pour moi un cadre idéal pour confronter mes connaissances théoriques à une problématique industrielle concrète, depuis la conception jusqu'au déploiement sur matériel réel. Mon objectif était de renforcer mes compétences en intelligence artificielle embarquée et de mieux comprendre les contraintes propres au développement de solutions destinées à un environnement de production.

La problématique confiée durant ce stage porte sur la surveillance automatisée de postes de travail industriels : comment détecter, en temps réel et à partir d'une simple caméra, les gestes d'un opérateur afin de reconnaitre automatiquement les différentes phases d'un cycle de travail (prise de la pièce, travail, dépose) et d'en mesurer la durée ? Une réponse fiable à cette question permettrait de disposer d'indicateurs de production objectifs, sans intervention manuelle, et de faciliter la détection des écarts de cadence : un enjeu central pour toute démarche d'amélioration continue en contexte Industrie 4.0.

Pour répondre à cette problématique, la démarche adoptée combine la vision par ordinateur (détection et suivi des mains avec MediaPipe, classification de gestes par un modèle MobileNetV2, détection d'objets avec EfficientDet-Lite0) et le déploiement embarqué sur des plateformes à ressources limitées (Raspberry Pi 4 et Jetson Nano), le tout restitué via un tableaude bord web permettant le suivi et l'historisation des cycles. Le développement a été mené de manière incrémentale, chaque brique étant validée avant l'ajout de la suivante.

Ce rapport présente, dans un premier chapitre, l'organisme d'accueil ainsi que la problématique traitée. Le deuxième chapitre est consacré à l'analyse des besoins et à la conception de la solution. Le troisième chapitre détaille la réalisation technique et les résultats obtenus, avant de conclure sur le bilan de cette expérience et les compétences acquises.

Mots clés : Industrie 4.0 / Vision par ordinateur / Systèmes embarqués / Détection de gestes / Surveillance de production

Chapitre 1 Organisme d'accueil et problématique traitée

1.1. Introduction

Ce premier chapitre a pour objectif de présenter le cadre général de mon stage d'été. Il décrit dans un premier temps l'organisme d'accueil, Novation City, son secteur d'activité et son organisation, avant de détailler la problématique industrielle qui m'a été confiée ainsi que le travail demandé. La démarche générale adoptée pour y répondre est ensuite présentée, suivie du chronogramme des tâches réalisées tout au long du stage. Le chapitre se termine par une synthèse des principales observations tirées de cette première phase d'immersion.

1.2. Présentation de l'entreprise

Novation City, anciennement dénommée « Pôle de Compétitivité de Sousse » avant sa refonte en avril 2018, est une société de droit privé créée en 2009 sur la base d'un partenariat public-privé. Elle est répartie sur deux sites, à Sousse et à Enfidha, pour une superficie totale d'environ 250 hectares, et se structure autour de trois « cités » thématiques dédiées respectivement à la mécatronique, au business et à l'industrie.
N'étant pas une entreprise commerciale au sens classique mais une structure d'accompagnement à l'innovation et à la compétitivité industrielle, Novation City ne communique pas de chiffre d'affaires ni d'actionnariat au sens d'une société de production : son fonctionnement repose sur un financement mixte public-privé et sur des partenariats institutionnels (Ministère de l'Industrie et des PME, coopération allemande GIZ, institut Fraunhofer, etc.). Son écosystème regroupe notamment une école d'ingénieurs, un centre de recherche en microélectronique et nanotechnologie, un centre de ressources technologiques, une université spécialisée en intelligence artificielle, une école doctorale, ainsi qu'une pépinière d'entreprises innovantes, animés par des équipes pluridisciplinaires d'ingénieurs, de chercheurs et de formateurs.

C'est dans ce cadre qu'a été lancé, le 18 septembre 2019, le Novation Industry 4.0 Center (anciennement CIDCI : Centre d'Innovation et de Développement des Compétences en Industrie 4.0), en partenariat avec la coopération allemande et sous l'égide du Ministère de l'Industrie et des PME, dans le cadre de l'initiative nationale « Vers une Industrie 4.0 en Tunisie ». C'est au

sein de cette entité, spécialisée dans l'accompagnement des entreprises industrielles tunisiennes vers la transformation digitale, que mon stage a été réalisé.

1.2.1. Produits et services

En tant que centre de compétences, Novation City ne commercialise pas de produits mais propose un ensemble de services orientés vers l'accompagnement des industriels dans leur transition vers l'Industrie 4.0, parmi lesquels :

• Le diagnostic de maturité numérique des entreprises industrielles

• L'accompagnement à la transition vers l'Industrie 4.0 et la mise en relation avec des fournisseurs de solutions technologiques

• Des actions de formation et de sensibilisation (ateliers, séminaires, événements)

• Des plateformes de démonstration et de prototypage technologique (preuves de concept, showroom Industrie 4.0)

• L'incubation d'entreprises innovantes via sa pépinière

• Des activités de recherche appliquée en microélectronique, nanotechnologie, intelligence artificielle et mécatronique.

1.2.2. Organisation générale de l'entreprise

Novation City est organisée autour d'une direction générale coiffant les trois cités thématiques évoquées précédemment, chacune régissant des pôles ou centres spécialisés. Le schéma simplifié ci dessous situe le Novation Industry 4.0 Center, cadre de mon stage, au sein de cette organisation :

• Direction générale de Novation City

– Cité de la Mécatronique (école d'ingénieurs, centres de recherche en microélectronique et nanotechnologie)

– Cité Business (pépinière d'entreprises, partenariats)

– Cité de l'Industrie : Novation AI center et Industry 4.0 Center c'est le cadre du stage : accompagnement des industriels, formation, démonstrateurs technologiques

1.3. Problématique traitée et travail demandé

Sur de nombreux postes de travail industriels, le suivi des cycles opérateur (prise de la pièce, exécution de la tâche, dépose) est encore réalisé de façon manuelle, par observation directe ou par chronométrage ponctuel. Cette approche présente plusieurs limites : elle est chronophage, sujette à des erreurs de mesure, difficilement généralisable à plusieurs postes simultanément, et ne permet pas un suivi continu ni une remontée automatique des indicateurs de performance (temps de cycle, cadence, dérives par rapport à un standard).

Le travail qui m'a été confié consiste à concevoir un système de vision par ordinateur capable de détecter automatiquement, à partir d'une caméra placée au-dessus d'un poste de travail, les gestes de la main de l'opérateur et d'en déduire la phase du cycle en cours, afin de mesurer automatiquement la durée de chaque étape. Le cahier des charges peut être résumé par les exigences suivantes :

• détecter la main de l'opérateur et sa position dans une grille de zones définies au-dessus du poste de travail
• identifier automatiquement le début d'un cycle (détection de la pièce dans la zone de prise)
• suivre l'enchaînement des étapes du cycle (prise → travail → dépose) via une machine à états
• fonctionner en temps réel sur du matériel embarqué à faible coût (Raspberry Pi 4, puis Jetson Nano) ;
• historiser les cycles réalisés et les restituer via un tableau de bord web consultable à distance
• rester simple à déployer et à maintenir, sans nécessiter d'infrastructure lourde côté poste de travail

1.4. Démarche-Méthodologie utilisée

Plutôt que d'entraîner un modèle de vision par ordinateur entièrement spécifique dès cette phase, la démarche retenue s'appuie sur des modèles pré-entraînés, éprouvés et légers, adaptés à une exécution en temps réel sur du matériel embarqué : la bibliothèque MediaPipe pour la détection et le suivi des mains, et le modèle EfficientDet-Lite0 pour la détection d'objets. Cette approche permet de limiter la complexité de mise en œuvre et le besoin en données d'entraînement, tout en offrant des performances suffisantes pour la problématique traitée.

Concrètement, la démarche se décompose comme suit :

• MediaPipe Hands est utilisé pour détecter la main de l'opérateur dans le flux vidéo et extraire ses 21 points de repère (landmarks), la position de la main est ensuite classifiée dans une grille

de zones (3×3) définie sur l'image, ce qui permet de savoir si la main se trouve dans la zone de prise, de travail ou de dépose

• le modèle EfficientDet-Lite0, entraîné sur les classes COCO, est utilisé comme condition de démarrage de cycle : il permet de détecter la présence d'une pièce dans la zone de prise avant de considérer qu'un nouveau cycle commence, ce qui limite les faux déclenchements

• les informations issues de ces deux modèles alimentent une machine à états (IDLE → PIECE_DETECTEE → EN_COURS_TRAVAIL → TERMINE) qui structure la logique de suivi du cycle et permet de mesurer la durée de chaque phase

• le développement a été mené de manière incrémentale : chaque brique (détection de la main, zones, détection d'objet, machine à états, interface) a été testée et validée séparément avant son intégration dans le pipeline complet, puis déployée d'abord sur Raspberry Pi 4 avant d'être portée sur Jetson Nano.

Cette méthodologie progressive, validée étape par étape, a permis de sécuriser chaque avancée technique avant de passer à la suivante, en particulier lors des phases de portage vers un nouveau matériel embarqué, souvent sources de contraintes de compatibilité logicielle.

1.5. Chronogramme des tâches menées durant le stage

Le tableau ci-dessous présente, à titre indicatif, la répartition par semaine des principales tâches réalisées durant le stage. Il pourra être ajusté pour correspondre précisément aux dates effectives du stage.

Tâche	**Sem. 1**	**Sem. 2**	**Sem. 3**	**Sem. 4**	**Sem. 5**	**Sem. 6**	
Accueil et découverte du centre	**X**						
Étude bibliographique (MediaPipe, TFLite, EfficientDet-Lite0)	**X**	**X**					
Prise en main de MediaPipe Hands et grille de zones		**X**					
Intégration d'EfficientDet-Lite0 (détection de la pièce)		**X**	**X**				
Conception du pipeline et de la machine à états			**X**				
Développement et tests sur Raspberry Pi 4			**X**	**X**			

Développement du tableau de bord Flask				**X**	**X**		
Portage et déploiement sur Jetson Nano					**X**	**X**	
Tests, validation et rédaction du rapport						**X**	**X**

1.6. Conclusion : Principales observations et constations

Cette première phase du stage a permis de confirmer l'intérêt d'une approche de suivi automatisé des postes de travail : le suivi manuel actuellement en place ne permet ni la continuité, ni la précision requises pour alimenter des indicateurs de performance fiables. Elle a également montré qu'il n'était pas nécessaire de développer des modèles de vision par ordinateur entièrement sur mesure pour obtenir un premier système fonctionnel : la combinaison de MediaPipe et d'EfficientDet-Lite0, deux modèles pré-entraînés et légers, constitue une base solide et rapide à mettre en œuvre pour répondre à la problématique posée, tout en restant compatible avec des contraintes de coût et de ressources embarquées limitées.

Ces constats orientent directement la suite du travail, détaillée dans le chapitre suivant, consacré à l'analyse des besoins et à la conception détaillée de la solution

Chapitre 2 : Analyse des besoins et conception

2.1. Introduction

Après avoir présenté, dans le chapitre précédent, l'organisme d'accueil et la problématique du suivi automatisé des postes de travail, ce chapitre a pour objectif d'analyser cette problématique plus en détail, avant de proposer une conception cohérente de la solution. Il s'ouvre sur une étude de l'existant, portant à la fois sur les pratiques actuelles de suivi des cycles opérateur et sur les approches technologiques disponibles pour y répondre. Les besoins fonctionnels et non fonctionnels attendus du système sont ensuite spécifiés, avant de présenter la conception retenue : architecture générale, machine à états et organisation du tableau de bord. Le chapitre se conclut par un retour critique sur cette phase d'analyse et de conception.

2.2. Etude de l'existant

Le suivi des cycles de travail sur les postes visités repose aujourd'hui essentiellement sur l'observation directe et le relevé manuel des temps, éventuellement complété par des feuilles de suivi papier renseignées par les opérateurs ou les superviseurs. Cette pratique, bien qu'utile, reste ponctuelle : elle ne couvre qu'un échantillon limité de cycles, dépend de la disponibilité d'un observateur, et n'offre pas de traçabilité continue des performances du poste.
Au-delà de cette pratique locale, plusieurs familles d'approches existent pour automatiser ce type de suivi, chacune présentant des avantages et des limites au regard du contexte du stage (poste de travail existant, contraintes de coût, absence d'infrastructure lourde). Le tableau suivant en dresse une synthèse comparative :

Approche	**Principe**	**Limites pour ce projet**
Chronométrage manuel (observation directe)	Un opérateur ou un superviseur mesure les temps de cycle à l'aide d'un chronomètre ou d'une feuille de relevé	Chronophage, sujet à l'erreur humaine, non généralisable à plusieurs postes en continu
Capteurs / instrumentation du poste (RFID, contacteurs, cellules photoélectriques)	Détection du passage de la pièce par des capteurs physiques installés sur le poste	Nécessite de modifier/instrumenter chaque poste, coût d'installation et de câblage, peu flexible

Dispositifs portés (wearables, gants à capteurs IMU)	Capteurs de mouvement portés par l'opérateur pour détecter ses gestes	Intrusif et contraignant pour l'opérateur, gêne au poste, acceptabilité limitée
Solutions de vision industrielle commerciales	Systèmes propriétaires de reconnaissance d'action basés sur du deep learning avec caméras dédiées	Coût élevé (licences, matériel, GPU serveur), déploiement lourd, peu adapté à un contexte de PME
Vision par ordinateur à base de modèles légers pré-entraînés (approche retenue)	Détection de mains et d'objets avec des modèles open-source optimisés pour l'embarqué (MediaPipe, EfficientDet-Lite0)	Nécessite un calibrage des zones par poste ; performances dépendantes des conditions d'éclairage et d'occlusion

Cette étude de l'existant met en évidence l'intérêt d'une approche par vision par ordinateur s'appuyant sur des modèles pré-entraînés et légers : elle évite l'instrumentation physique du poste, ne contraint pas l'opérateur, et reste compatible avec un déploiement sur du matériel embarqué à faible coût, contrairement aux solutions industrielles propriétaires généralement plus onéreuses et plus complexes à intégrer.

2.3. Analyse (ou spécification) des besoins

2.3.1. Besoins fonctionnels

Le système à concevoir doit répondre aux besoins fonctionnels suivants :

- détecter en temps réel la présence et la position de la main de l'opérateur dans le champ de la caméra
- classifier la position de la main selon une grille de zones prédéfinies (zone de prise, zone de travail, zone de dépose)
- détecter automatiquement la présence d'une pièce dans la zone de prise, afin de déclencher le début d'un nouveau cycle
- suivre l'enchaînement des phases du cycle à l'aide d'une machine à états, et calculer la durée de chaque phase
- historiser chaque cycle réalisé (horodatage, durées, statut)
- afficher un flux vidéo en direct ainsi que l'état courant du poste sur un tableau de bord web
- permettre la consultation de l'historique des cycles (tableau, visualisation type Gantt) et, si nécessaire, une validation manuelle.

2.3.2. Besoins non fonctionnels

En complément, plusieurs exigences non fonctionnelles ont été identifiées :

• performance temps réel : traitement du flux vidéo à une cadence suffisante pour ne pas manquer une transition d'état (de l'ordre de quelques images par seconde minimum) ;

• légèreté et faible coût : fonctionnement sur du matériel embarqué grand public (Raspberry Pi 4, Jetson Nano), sans GPU dédié ni licence propriétaire

• simplicité de déploiement et de maintenance : installation via des environnements Python isolés (conda/Miniforge), sans dépendance à une infrastructure serveur externe

• robustesse raisonnable aux variations de luminosité et à de légères occlusions de la main

• interface claire et accessible depuis un simple navigateur web, sans installation côté utilisateur.

2.3.3. Acteurs du système

Trois acteurs principaux interagissent avec le système :

• l'opérateur, dont les gestes sont observés par la caméra, sans action particulière de sa part

• le superviseur / responsable de production, qui consulte le tableau de bord pour suivre les cycles et les indicateurs de performance

• le système lui-même, qui assure de façon autonome l'acquisition vidéo, la détection, la logique de cycle et la journalisation.

2.4. Conception

2.4.1. Architecture générale

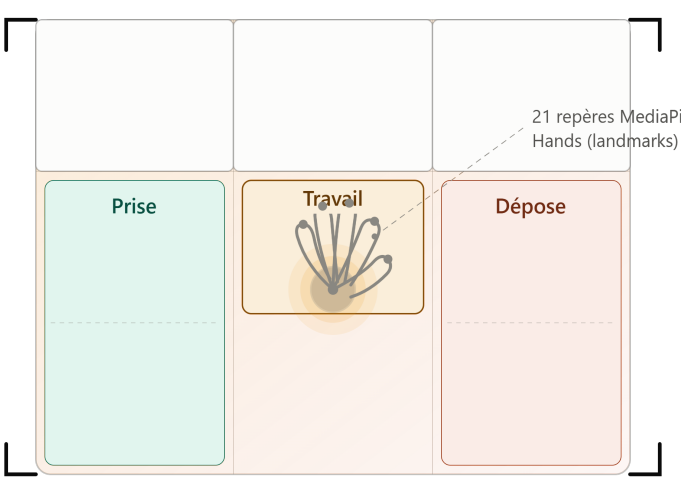
L'architecture retenue s'organise autour d'un pipeline séquentiel, illustré ci-dessous :

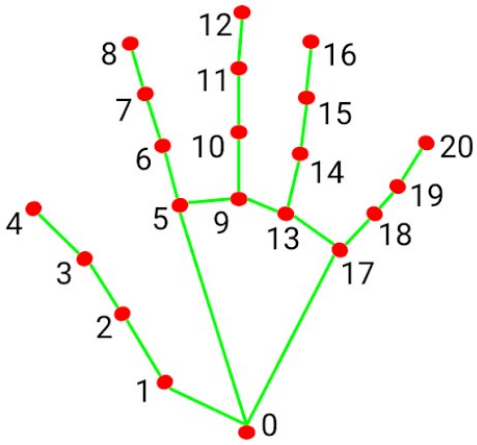


Le flux vidéo est capturé par la caméra puis prétraité (redimensionnement, conversion de format) avant d'être soumis en parallèle aux deux modèles de vision par ordinateur : MediaPipe Hands pour la détection et le suivi de la main, et EfficientDet-Lite0 pour la détection de la pièce. Les résultats de ces deux modèles alimentent la machine à états, qui détermine la phase du cycle en cours. Chaque transition d'état est historisée et transmise au tableau de bord web, développé avec Flask, qui affiche le flux vidéo annoté ainsi que l'historique des cycles.

2.4.2. Détection des mains et grille de zones

MediaPipe Hands fournit, pour chaque image, les 21 points de repère (landmarks) de la main détectée. À partir de ces points, un point de référence (par exemple le centre de la paume) est projeté sur une grille de zones définie sur l'image, généralement une grille 3×3, dont chaque cellule correspond à une fonction du poste (zone de prise, zone de travail, zone de dépose, zones neutres). Cette approche évite d'avoir à entraîner un modèle spécifique de détection de zone : elle repose uniquement sur une règle géométrique appliquée à la sortie du modèle pré-entraîné.





Rangée du haut neutre ; la main est projetée sur les 3 zones fonctionnelles ci-dessus.

2.4.3. Machine à états du cycle

La logique de suivi du cycle est modélisée par une machine à états à quatre états, détaillée dans le tableau suivant :



État	Condition d'entrée	Action réalisée	Transition vers
IDLE	État initial / fin de cycle précédent	Attente d'une pièce dans la zone de prise	PIECE_DETECTEE
PIECE_DETECTEE	EfficientDet-Lite0 détecte une pièce dans la zone de prise	Horodatage du début de cycle	EN_COURS_TRAVAIL
EN_COURS_TRAVAIL	La main de l'opérateur est suivie dans la zone de travail	Mesure du temps écoulé dans la zone de travail	TERMINE
TERMINE	La main quitte la zone de travail vers la zone de dépose	Calcul des durées, enregistrement du cycle (JSON), mise à jour du tableau de bord	IDLE

2.5. Conclusion

Cette phase d'analyse et de conception a permis de transformer la problématique générale posée en chapitre 1 en un ensemble de besoins précis, puis en une architecture concrète et réalisable avec des ressources limitées. Elle confirme la pertinence du choix initial de s'appuyer sur des modèles pré-entraînés plutôt que sur un développement de modèles propres à ce stade : la combinaison de MediaPipe et d'EfficientDet-Lite0 couvre l'essentiel des besoins fonctionnels identifiés, sans complexifier inutilement le projet.

Cette étape a néanmoins mis en lumière certaines limites intrinsèques à l'approche retenue, qu'il conviendra de garder à l'esprit lors de la réalisation : la détection par grille de zones reste sensible au positionnement et à l'angle de la caméra, ce qui impose un calibrage propre à chaque poste ; la robustesse aux occlusions partielles de la main n'a pas encore été testée en conditions réelles ; enfin, l'usage de modèles génériques (non ré-entraînés sur les gestes spécifiques du poste étudié) constitue une simplification volontaire qui pourra, si nécessaire, être dépassée dans une itération future par un modèle de classification de gestes dédié. Ces constats n'invalident pas

la conception proposée, mais en délimitent clairement le périmètre et annoncent les points de vigilance à traiter lors de la phase de réalisation, objet du chapitre suivant.

Chapitre 3 : Réalisation

3.1. Introduction

Ce chapitre présente la mise en œuvre concrète de la conception décrite au chapitre précédent. Il détaille les choix technologiques et les environnements de développement utilisés, les fonctionnalités effectivement réalisées, l'interface du tableau de bord obtenue, ainsi que les tests et validations menés sur les deux plateformes matérielles ciblées : Raspberry Pi 4 et Jetson Nano. Il se termine par un bilan critique de cette étape de réalisation, qui constitue le cœur technique du stage.

3.2. Choix technologiques et environnements de développement

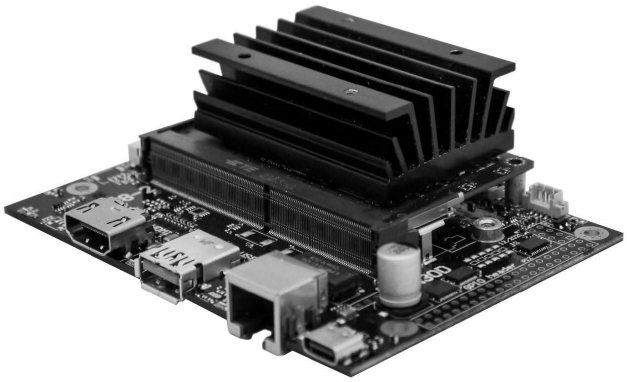
Le développement a été réalisé en Python, choisi pour la richesse de son écosystème en vision par ordinateur et sa portabilité entre les deux plateformes embarquées ciblées. Deux environnements ont été mis en place et validés, l'un sur Raspberry Pi 4, l'autre sur Jetson Nano (carte porteuse Auvidea JN30D), résumés dans le tableau suivant :

Élément	Raspberry Pi 4	Jetson Nano (carte Auvidea JN30D)
Système d'exploitation	Raspberry Pi OS Bookworm 64 bits	L4T R32.7.1 / JetPack 4.6.x, Ubuntu 18.04 aarch64 (BSP Auvidea)
Gestion des environnements	Miniforge/conda (environnement « gesture_env »)	Miniforge/conda (environnement dédié, hooks LD_LIBRARY_PATH / LD_PRELOAD permanents)
Bibliothèques clés	mediapipe 0.10.18, opencv-python 4.10.0.84, numpy 1.26.4, Flask	mêmes versions épinglées, pour compatibilité aarch64 / GLIBCXX (Ubuntu 18.04)
Caméra	Webcam USB REDRAGON	Webcam USB REDRAGON, branchée directement (pas de hub)

Interface de suivi	Tableau de bord Flask (app.py, flux MJPEG)	Tableau de bord Flask (app.py, flux MJPEG)
Accélération matérielle	CPU uniquement (Cortex-A72)	CPU (Cortex-A57) GPU

3.2.1. Présentation des cartes utilisées

Le choix s'est porté sur deux cartes embarquées grand public, largement utilisées dans les projets d'Industrie 4.0 pour leur bon rapport performance/coût : le Raspberry Pi 4 Model B, retenu comme plateforme de référence, et le Jetson Nano de NVIDIA, monté ici sur une carte porteuse tierce (Auvidea JN30D), retenu pour son GPU embarqué et son potentiel d'accélération, encore inexploité à ce stade du projet. Leurs principales caractéristiques sont résumées ci-dessous :





Caractéristique	Raspberry Pi 4 Model B	Jetson Nano (module + carte Auvidea JN30D)
Processeur (SoC)	Broadcom BCM2711, quad-core ARM Cortex-A72 64 bits, ≈ 1,5 GHz	Quad-core ARM Cortex-A57 64 bits, ≈ 1,43 GHz
GPU	Broadcom VideoCore VI	128 cœurs NVIDIA Maxwell (CUDA), non exploités dans le projet actuel
Mémoire vive	Jusqu'à 4 Go LPDDR4 (selon la version)	4 Go LPDDR4

Stockage	Carte microSD (externe)	16 Go eMMC embarqué sur le module, porté par la carte Auvidea JN30D
Vidéo / caméra	2 × micro-HDMI (jusqu'à 4K)	HDMI 2.0 ; connecteur caméra MIPI CSI-2 (non utilisé ici, webcam USB privilégiée)
Réseau / USB	Gigabit Ethernet, Wi-Fi 802.11ac, Bluetooth 5.0, 2 × USB 3.0, 2 × USB 2.0	Gigabit Ethernet et USB 3.0, exposés via la carte porteuse Auvidea
GPIO	40 broches	40 broches, exposées via la carte porteuse
OS utilisé dans le projet	Raspberry Pi OS Bookworm 64 bits	L4T R32.7.1 / JetPack 4.6.x, Ubuntu 18.04 aarch64 (BSP Auvidea)
Rôle dans le projet	Plateforme de référence, validée en premier	Plateforme cible secondaire, pour évaluer portabilité et potentiel GPU

Le Jetson Nano se distingue par la présence d'un GPU dédié (128 cœurs CUDA), absent du Raspberry Pi 4, ce qui en fait a priori une plateforme plus adaptée à l'inférence de modèles de vision par ordinateur. Dans la pratique cependant, ce potentiel reste conditionné à une conversion explicite des modèles vers TensorRT (voir section 3.5) : sans cette conversion, les deux cartes exécutent l'inférence sur CPU, et les cœurs Cortex-A72 du Raspberry Pi 4, plus récents, se sont montrés au moins aussi rapides que les cœurs Cortex-A57 du Jetson Nano sur les tests réalisés. Le choix d'une carte porteuse tierce (Auvidea JN30D) pour le Jetson Nano a par ailleurs introduit des contraintes supplémentaires par rapport à un kit de développement NVIDIA standard, notamment l'obligation d'utiliser le BSP du fabricant de la carte plutôt que l'image JetPack officielle

3.2.2. Gestion des dépendances et épinglage des versions

Sur les deux plateformes, la gestion des dépendances repose sur des environnements Conda isolés (Miniforge), avec un épinglage strict des versions des bibliothèques critiques. Ce choix n'est pas arbitraire : sur le Jetson, dont le système d'exploitation (Ubuntu 18.04 aarch64) est ancien, les versions récentes de MediaPipe et de NumPy provoquent des erreurs de compatibilité liées à la bibliothèque GLIBCXX. L'environnement type est créé de la façon suivante :

```
conda create -n gesture_env python=3.11
conda activate gesture_env
pip install mediapipe==0.10.18 \
            numpy==1.26.4 \
            opencv-python==4.10.0.84 \
            flask ai-edge-litert
```

Sur le Jetson Nano, deux variables d'environnement supplémentaires doivent être exportées à chaque activation pour que MediaPipe et OpenCV trouvent correctement leurs bibliothèques partagées. Plutôt que de les redéfinir manuellement à chaque session, elles ont été intégrées à un hook d'activation permanent de l'environnement Conda :

```
# fichier: $CONDA_PREFIX/etc/conda/activate.d/env_vars.sh
export LD_LIBRARY_PATH=$CONDA_PREFIX/lib:$LD_LIBRARY_PATH
export LD_PRELOAD=$CONDA_PREFIX/lib/libgomp.so.1
```

Le portage sur le Jetson a par ailleurs nécessité l'utilisation du BSP (Board Support Package) fourni par Auvidea plutôt que l'image JetPack officielle de NVIDIA, incompatible avec cette carte porteuse spécifique.

3.2.3. Outils et connectivité de développement

Le développement s'est appuyé sur une connexion SSH via PuTTY pour piloter les cartes à distance, des transferts de fichiers via FileZilla (protocole SFTP), et une connexion Ethernet directe entre le PC hôte et la carte cible, configurée via le partage de connexion Windows (Internet Connection Sharing), le PC étant adressé en 192.168.137.1 et la carte cible en 192.168.137.50.

3.3. Les fonctionnalités réalisées

Le tableau suivant récapitule les fonctionnalités effectivement développées et validées à l'issue du stage, la technologie associée, et le résultat obtenu :

Fonctionnalité	Composant / technologie	Résultat
Acquisition vidéo	OpenCV (VideoCapture)	Flux capturé et prétraité en continu depuis la webcam USB
Détection de la main	MediaPipe Hands	21 repères extraits par image, projetés sur la grille de zones
Détection de la pièce	EfficientDet-Lite0 (TFLite)	Déclenchement fiable du début de cycle dans la zone de prise

Logique de cycle	Machine à états (Python)	4 états, horodatage et calcul automatique des durées
Diffusion vidéo	Flask + MJPEG	Flux annoté visible en direct dans le navigateur
Historisation	Fichier JSON	Cycles enregistrés avec horodatage, durées et statut
Visualisation	Flask + tableau + Gantt	Historique consultable, cycles représentés graphiquement
Déploiement	Miniforge/conda (Pi 4 et Jetson)	Pipeline fonctionnel identique sur les deux plateformes

3.3.1. Détection et suivi de la main

MediaPipe Hands est appliqué à chaque image du flux vidéo prétraité. Lorsqu'une main est détectée, ses 21 repères (landmarks) sont extraits ; un point de référence dérivé de ces repères (centre de la paume) est ensuite projeté sur la grille de zones définie sur l'image, ce qui permet de déterminer en continu dans quelle zone fonctionnelle (prise, travail ou dépose) se trouve la main de l'opérateur.

3.3.2. Détection de la pièce et déclenchement de cycle

Le modèle EfficientDet-Lite0, exécuté via le runtime TensorFlow Lite, est utilisé pour détecter la présence d'un objet dans la zone de prise. Cette détection constitue la condition de démarrage d'un nouveau cycle : elle évite qu'un simple passage de la main dans la zone de prise, sans pièce réelle, ne déclenche à tort un nouveau cycle.

3.3.3. Machine à états et journalisation

La logique de cycle est implémentée sous la forme d'une machine à états explicite en Python, dont chaque transition est horodatée. À la clôture d'un cycle (état TERMINE), les durées de chaque phase sont calculées puis écrites dans un fichier d'historique au format JSON, immédiatement exploitable par le tableau de bord.

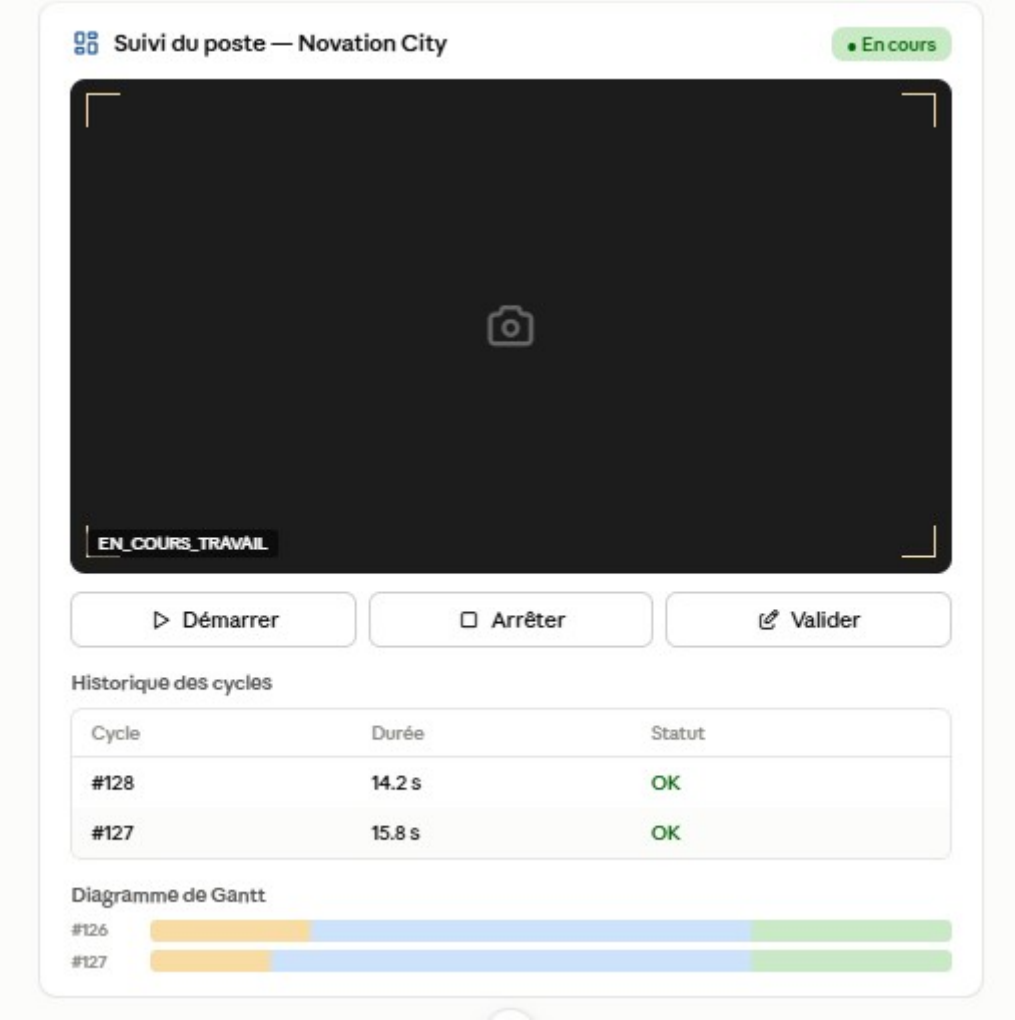
3.3.4. Tableau de bord et déploiement multi-plateforme

Le tableau de bord, développé avec Flask, diffuse le flux vidéo annoté en direct (streaming MJPEG) et expose l'historique des cycles sous forme de tableau et de diagramme de Gantt. L'ensemble du pipeline : capture, détection, machine à états, tableau de bord a été déployé et

validé fonctionnellement à l'identique sur les deux plateformes cibles, Raspberry Pi 4 et Jetson Nano.

3.4. Les interfaces / le prototype

Le tableau de bord, accessible depuis un navigateur sur le réseau local, s'organise autour des blocs suivants :



Le parcours d'utilisation est volontairement simple : le superviseur ouvre le tableau de bord dans son navigateur, démarre le suivi, observe le flux vidéo annoté en direct pendant que les cycles s'enchaînent automatiquement, et peut à tout moment consulter l'historique ou intervenir manuellement en cas de détection ambiguë via la fenêtre de validation.

3.5. Tests et simulations

Le fonctionnement du pipeline a été validé étape par étape, conformément à la démarche incrémentale annoncée au chapitre 1. Sur le Jetson Nano, l'installation de MediaPipe a été

confirmée fonctionnelle (import réussi, version 0.10.18), de même que la détection de la webcam sur /dev/video. Sur le Raspberry Pi 4, l'ensemble du tableau de bord a été testé fonctionnellement : démarrage/arrêt du suivi, journalisation des cycles, affichage du diagramme de Gantt, historique et validation manuelle.

Cette phase de tests a également permis d'identifier et de résoudre plusieurs problèmes techniques, résumés dans le tableau suivant :

Problème rencontré	Cause identifiée	Solution appliquée
Erreurs d'énumération USB (-110 / -71) sur Jetson Nano	Caméra branchée via un hub USB	Connexion directe de la caméra à un port USB natif de la carte
Échecs d'installation / incompatibilités MediaPipe sur le Jetson	Versions récentes de mediapipe/numpy non compatibles avec Ubuntu 18.04 aarch64 (GLIBCXX)	Épinglage des versions (mediapipe==0.10.18, numpy==1.26.4, opencv-python==4.10.0.84)
TypeError lors du passage d'images recadrées à mp.Image()	Tableaux numpy non contigus après recadrage (crop)	Application systématique de np.ascontiguousarray() avant l'appel
Déconnexions SSH répétées pendant les sessions de développement	Mise en veille de l'adaptateur réseau Windows et réinitialisation de la case ICS après veille	Désactivation de la mise en veille de l'adaptateur, réactivation manuelle de l'ICS à chaque session
Absence de gain de performance sur le Jetson malgré son GPU	Inference MediaPipe/OpenCV exécutée uniquement sur CPU (XNNPACK), GPU non sollicité	Identifié comme axe d'amélioration futur (conversion des modèles en TensorRT)

Sur le plan des performances, les traitements s'exécutent actuellement uniquement sur CPU, sur les deux plateformes : ni MediaPipe (mode XNNPACK), ni OpenCV standard ne sollicitent le GPU du Jetson Nano par défaut. Il a également été observé que les cœurs Cortex-A57 du Jetson Nano peuvent, à fréquence égale, être plus lents par cœur que les Cortex-A72 du Raspberry Pi 4, de sorte qu'aucun gain de performance n'est garanti sur le Jetson sans conversion explicite des modèles vers TensorRT.

3.6. Conclusion

Cette phase de réalisation a permis d'aboutir à un pipeline fonctionnel de bout en bout, depuis l'acquisition vidéo jusqu'à la restitution des cycles sur un tableau de bord web, déployé avec succès sur deux plateformes embarquées aux contraintes différentes. Elle confirme la viabilité de l'approche retenue au chapitre 2, fondée sur des modèles pré-entraînés légers

(MediaPipe, EfficientDet-Lite0) plutôt que sur un développement de modèles propres, y compris dans un contexte de ressources embarquées limitées.

Cette étape a également mis en évidence l'importance des contraintes de bas niveau, souvent sous-estimées dans la conception : compatibilité fine des versions de bibliothèques sur une plateforme ancienne (Ubuntu 18.04 aarch64), gestion du matériel USB, configuration réseau instable côté PC hôte, ou encore absence d'accélération GPU par défaut. La résolution méthodique de ces problèmes, documentée tout au long du développement, a été aussi formatrice que le développement fonctionnel lui-même. Elle ouvre par ailleurs des pistes claires pour la suite du projet, notamment la conversion des modèles vers TensorRT pour exploiter le GPU du Jetson, et l'intégration future d'un modèle de classification de gestes dédié.

Conclusion générale

Ce stage, réalisé à Novation City (Centre d'Excellence en Industrie 4.0, Sousse), visait à automatiser le suivi des cycles opérateur (prise, travail, dépose) sur les postes industriels, aujourd'hui assuré manuellement, donc chronophage et peu précis. La démarche retenue a combiné deux modèles pré-entraînés légers, MediaPipe Hands et EfficientDet-Lite0, au sein d'une machine à états, déployés de façon incrémentale sur Raspberry Pi 4 et Jetson Nano, et restitués via un tableau de bord web développé avec Flask.

Les fonctionnalités principales ont été réalisées : détection et suivi de la main, détection de la pièce, machine à états complète, streaming vidéo et tableau de bord (historique, validation manuelle, Gantt), déployés à l'identique sur les deux plateformes. La classification fine des gestes (modèle MobileNetV2) a été entraînée séparément mais pas encore intégrée, et l'accélération GPU du Jetson via TensorRT n'a pas été mise en œuvre, faute de temps. La solution reste perfectible : la grille de zones, calibrée manuellement, est sensible au positionnement de la caméra, les modèles génériques limitent la finesse de détection, et les tests sont restés qualitatifs plutôt que quantitatifs. Les prochaines étapes logiques sont la conversion TensorRT, l'intégration du classifieur de gestes, l'automatisation de la calibration et une validation prolongée en conditions réelles.

Ce stage m'a permis d'acquérir des compétences techniques concrètes (vision par ordinateur embarquée, déploiement Linux sur Raspberry Pi et Jetson, gestion d'environnements Python, développement web léger) ainsi que des compétences socioprofessionnelles (autonomie, rigueur, gestion du temps sur un projet multi-étapes). Le principal enseignement est que les contraintes de bas niveau (compatibilité logicielle, matériel, réseau) pèsent souvent plus lourd que la conception algorithmique elle-même. Cette expérience conforte mon intérêt pour les systèmes embarqués intelligents appliqués à l'industrie, un axe que je souhaite approfondir lors de mon stage de PFE, dans une entreprise ou un centre technologique engagé dans une démarche similaire de transformation digitale.

Mon bilan de compétences

Quels métiers d'ingénieurs" j’ai l’intention d’exercer dans le futur

Je souhaite exercer un métier d'ingénieur à l'interface entre l'électronique embarquée, la conception matérielle (PCB) et l'intelligence artificielle : par exemple ingénieur en systèmes embarqués intelligents, ingénieur R&D en IA embarquée, ou ingénieur en conception électronique intégrant des fonctions de traitement intelligent (vision par ordinateur, apprentissage automatique) directement sur la carte. Ce profil couvre toute la chaîne de la conception matérielle (schématique, routage PCB, choix des composants) jusqu'au déploiement de modèles d'IA sur des plateformes à ressources limitées une direction que ce stage à Novation City m'a permis de commencer à explorer concrètement.

Quels métiers d'ingénieurs" je ne souhaiterai pas exercer ?

Fiche bilan de compétences

Compétences acquises

Compétence	Degré de maîtrise (%)	Occasion de pratique
Compétences techniques pratiques		
Vision par ordinateur embarquée (MediaPipe, TensorFlow Lite)	70	Stage Novation City
Déploiement de systèmes embarqués Linux (Raspberry Pi OS, L4T/Jetson)	70	Stage Novation City
Gestion d'environnements Python et résolution de dépendances (conda/Miniforge)	80	Stage Novation City
Développement web léger (Flask, streaming vidéo)	40	Stage Novation City
Programmation Python appliquée (traitement d'image, machine à états)	70	Stage + cursus ENIS
Diagnostic et résolution de problèmes matériels/logiciels embarqués	80	Stage Novation City
Compétences socioprofessionnelles (soft skills)		
Autonomie dans la conduite d'un projet technique	80	Stage Novation City
Rigueur et méthode (démarche incrémentale, documentation)	70	Stage Novation City
Gestion du temps sur un projet multi-étapes	70	Stage Novation City
Communication écrite technique (rapport, documentation)	80	Stage + cursus ENIS
Capacité d'adaptation face à des blocages techniques imprévus	80	Stage Novation City

Compétences à développer

Priorité	Compétence ciblée	Moyen à utiliser (suggestion)
Compétences techniques pratiques		
1ère	Optimisation GPU / conversion de modèles (TensorRT, CUDA)	*PFE, formation en ligne, projet personnel*
1ère	Conteneurisation pour le déploiement embarqué (Docker)	*Auto-formation, projet personnel*
2e	Tests automatisés et intégration continue (CI)	*Projet module, PFE*
2e	Prototypage FPGA / usinage	*TP, formation*
Compétences socioprofessionnelles (soft skills)		
1ère	Communication orale et présentation technique (soutenances)	*Soutenances, clubs, challenges*
2e	Anglais technique à l'écrit et à l'oral	*auto-formation*

Références

https://www.youtube.com/watch?v=id2UDqA4X-4&list=WL&index=2&t=91s

https://www.nvidia.com/fr-fr/autonomous-machines/embedded-systems/

https://www.youtube.com/watch?v=DVfmw3Ftu3I&list=WL&index=5

https://forums.raspberrypi.com/viewtopic.php?t=388498

https://github.com/LinkedInLearning/computer-vision-3153829

https://thinkrobotics.com/blogs/tutorials/opencv-object-detection-with-raspberry-pi-complete-computer-vision-guide?srsltid=AfmBOopHBODWKCwxbcxBZsiUuZTep7gzBBuTScuqPGGg6Fxli6p57EyJ

https://www.projectpro.io/article/object-detection-project-ideas-with-source-code/490

https://www.nvidia.com/fr-fr/autonomous-machines/embedded-systems/jetson-nano/product-development/